

MILESTONE 3 — SPRINT 5

S5-EVENTS

Complete Study Guide

Payment Saga, Settlement Lifecycle, RabbitMQ Topology, Feign Integration & Unit Tests

SERVICE

booking-service (S5)

BRANCH

feat/M3/booking/S5-EVENTS

PATTERN

Choreography Saga

TESTS

59 passing / 0 failing

Table of Contents

1	Project Architecture Overview	
2	RabbitMQ Topology & AMQP Configuration	
	2.1 Exchanges & Queues	
	2.2 Dead-Letter Queue (DLQ)	
	2.3 AmqpConfig.java — full bean declarations	
3	Payment Event Publishers	
4	Contract Event Records (shared library)	
5	ItineraryEventConsumer — Saga Consumer	
	5.1 onItineraryCompleted flow	
	5.2 onItineraryCancelled flow	
6	Settlement Entity & Repository	
7	SettlementService — Full Idempotency Lifecycle	
8	BookingAggregateService — Aggregate Endpoints	
9	Feign Client Integration (S5-F3, F4, F10, F12)	
	9.1 S5-F3: User existence check	
	9.2 S5-F4: Itinerary validation + seasonal surcharge	
	9.3 S5-F10: Revenue by destination & season	
	9.4 S5-F12: Refund cancellation tier (atomic UPDATE)	
10	BookingRepository — Custom Queries	
11	Kubernetes Deployment & Observability	
12	Unit Tests — All 4 Test Classes	
13	End-to-End Saga Flow — Visual Summary	

Project Architecture Overview

The **Trip Planning Social Network** is a Spring Boot microservices application composed of 5 services. Each service owns its database and communicates via REST (Feign clients) and events (RabbitMQ). This guide focuses on the **booking-service** (S5) and the S5-EVENTS sprint work.

Service	Port	Database(s)	Role
user-service	8081	PostgreSQL, Neo4j	User accounts, social graph
itinerary-service	8082	PostgreSQL, MongoDB	Trips, destinations, activities
destination-service	8083	PostgreSQL	Destination catalog
notification-service	8084	MongoDB	Push/email notifications
booking-service	8085	PostgreSQL, MongoDB, Redis	Bookings, settlements, payments

Shared Contracts Maven Module

All services share a Maven module `com.team15.tripplanning:contracts:1.0-SNAPSHOT` that contains:

- **Event records** — `PaymentInitiatedEvent`, `PaymentCompletedEvent`, `PaymentFailedEvent`, `PaymentRefundedEvent`, `ItineraryCompletedEvent`, `ItineraryCancelledEvent`
- **DTOs** — `UserDTO`, `ItineraryDTO`, `ItinerarySummaryDTO`, `DestinationSummaryDTO`, `BatchItineraryRequest`, `BatchDestinationRequest`, `UserBookingTotalDTO`, `ConfirmedSummaryDTO`, `ItineraryAggregateDTO`, `BookingAggregateRequest`
- **Feign clients** — `UserServiceClient`, `ItineraryServiceClient`, `DestinationServiceClient`

WHY A SHARED MODULE?

So every service agrees on the exact same event payload shape and DTO structure at compile time. No guessing about field names — if the contracts module compiles, all services are in sync.

2.1 Exchanges & Queues

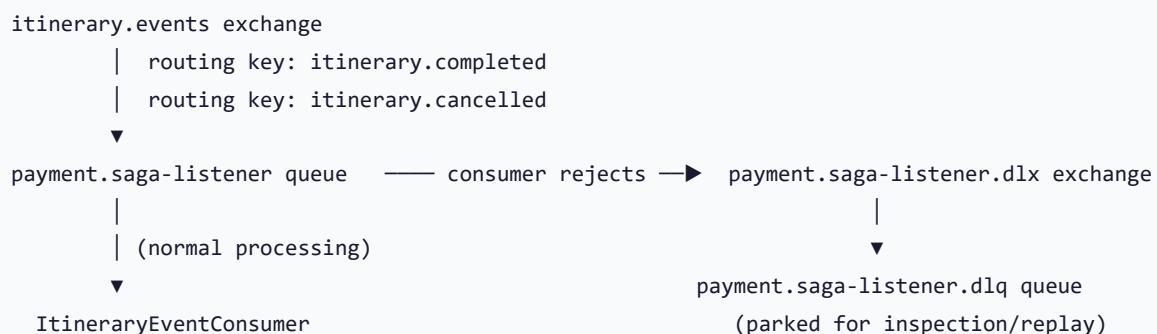
RabbitMQ uses a **TopicExchange** so routing keys can contain wildcards (`#` , `*`). Two exchanges are declared:

Exchange name	Type	Purpose
<code>payment.events</code>	topic	booking-service publishes payment lifecycle events TO other services
<code>itinerary.events</code>	topic	itinerary-service publishes trip lifecycle events; booking-service CONSUMES from this

Queue	Bound to exchange	Routing keys
<code>payment.saga-listener</code>	<code>itinerary.events</code>	<code>itinerary.completed</code> , <code>itinerary.cancelled</code>
<code>payment.saga-listener.dlq</code>	<code>payment.saga-listener.dlx</code>	(DLX/DLQ — receives rejected messages)

2.2 Dead-Letter Queue (DLQ) Pattern

When a message is rejected (or expires) from `payment.saga-listener` , RabbitMQ automatically re-routes it to a **Dead Letter Exchange (DLX)** which then delivers it to the DLQ for debugging/replaying.



2.3 AmqpConfig.java — Key Bean Declarations

```

// payment.events — booking-service publishes payment lifecycle events
@Bean TopicExchange paymentEventsExchange() {
    return new TopicExchange("payment.events");
}

// itinerary.events — booking-service listens for trip events
@Bean TopicExchange itineraryEventsExchange() {
    return new TopicExchange("itinerary.events");
}
  
```

```

}

// Main listener queue with DLX configured
@Bean Queue sagaListenerQueue() {
    return QueueBuilder.durable("payment.saga-listener")
        .withArgument("x-dead-letter-exchange", "payment.saga-listener.dlx")
        .build();
}

// Dead-letter infrastructure
@Bean DirectExchange sagaDlx() {
    return new DirectExchange("payment.saga-listener.dlx");
}
@Bean Queue sagaDlq() {
    return QueueBuilder.durable("payment.saga-listener.dlq").build();
}
@Bean Binding dlqBinding() {
    return BindingBuilder.bind(sagaDlq()).to(sagaDlx())
        .with("payment.saga-listener.dlq");
}

// Bindings: itinerary.events → payment.saga-listener
@Bean Binding bindCompleted() {
    return BindingBuilder.bind(sagaListenerQueue())
        .to(itineraryEventsExchange())
        .with("itinerary.completed");
}
@Bean Binding bindCancelled() {
    return BindingBuilder.bind(sagaListenerQueue())
        .to(itineraryEventsExchange())
        .with("itinerary.cancelled");
}

// JSON serializer – events sent/received as JSON objects, not Java serialized bytes
@Bean MessageConverter jsonMessageConverter() {
    return new Jackson2JsonMessageConverter();
}
@Bean RabbitTemplate rabbitTemplate(ConnectionFactory cf) {
    RabbitTemplate t = new RabbitTemplate(cf);
    t.setMessageConverter(jsonMessageConverter());
    return t;
}

```

AUTO-STARTUP FLAG

In `application.yml`, `spring.rabbitmq.listener.simple.auto-startup: ${M3_MESSAGING_ENABLED:false}` means the consumer does NOT start when running locally (no RabbitMQ server needed). In Kubernetes, the ConfigMap sets `M3_MESSAGING_ENABLED: "true"` to enable it.

Payment Event Publishers

`PaymentEventPublisher` is a Spring `@Service` that wraps `RabbitTemplate`. It provides four methods, one per payment lifecycle event. **All four send to the same exchange** (`payment.events`) but with different routing keys.

Method	Exchange	Routing Key	Triggered when
<code>publishPaymentInitiated</code>	<code>payment.events</code>	<code>payment.initiated</code>	itinerary.completed received → settlement created
<code>publishPaymentCompleted</code>	<code>payment.events</code>	<code>payment.completed</code>	SettlementService approves payment (amount > 0)
<code>publishPaymentFailed</code>	<code>payment.events</code>	<code>payment.failed</code>	SettlementService rejects payment (amount = 0)
<code>publishPaymentRefunded</code>	<code>payment.events</code>	<code>payment.refunded</code>	itinerary.cancelled → CONFIRMED booking cancelled

```
@Service
@RequiredArgsConstructor
public class PaymentEventPublisher {

    private final RabbitTemplate rabbitTemplate;
    private static final String EXCHANGE = "payment.events";

    public void publishPaymentInitiated(PaymentInitiatedEvent event) {
        rabbitTemplate.convertAndSend(EXCHANGE, "payment.initiated", event);
    }
    public void publishPaymentCompleted(PaymentCompletedEvent event) {
        rabbitTemplate.convertAndSend(EXCHANGE, "payment.completed", event);
    }
    public void publishPaymentFailed(PaymentFailedEvent event) {
        rabbitTemplate.convertAndSend(EXCHANGE, "payment.failed", event);
    }
    public void publishPaymentRefunded(PaymentRefundedEvent event) {
        rabbitTemplate.convertAndSend(EXCHANGE, "payment.refunded", event);
    }
}
```

WHY TOPIC EXCHANGE?

A TopicExchange lets other services subscribe with wildcard bindings (e.g., `payment.#`) to receive all payment events, or `payment.completed` to receive only the completed event. This is more flexible than Direct or Fanout exchanges.

Contract Event Records (Shared Library)

All events are Java `record` types in the contracts module. Records are immutable, have auto-generated constructors>equals/hashCode, and serialize cleanly to JSON via Jackson.

```
// Sent when an itinerary trip is completed (itinerary-service publishes this)
public record ItineraryCompletedEvent(
    Long itineraryId,
    Long organizerId,
    Long destinationId,
    BigDecimal totalAmount
) {}

// Sent when an itinerary is cancelled
public record ItineraryCancelledEvent(
    Long itineraryId,
    Long organizerId,
    Long destinationId,
    String reason
) {}

// booking-service publishes these four:
public record PaymentInitiatedEvent(Long settlementId, Long itineraryId, BigDecimal amount) {}
public record PaymentCompletedEvent(Long settlementId, Long itineraryId, BigDecimal amount) {}
public record PaymentFailedEvent(Long settlementId, Long itineraryId, String reason) {}
public record PaymentRefundedEvent(Long settlementId, Long itineraryId, BigDecimal refundAmount) {}
```

KEY INSIGHT: AMOUNT COMES FROM DB, NOT FROM THE EVENT

The `ItineraryCompletedEvent.totalAmount` field is present but the consumer does NOT use it for the settlement amount. Instead, it queries `SUM(amount) WHERE status='CONFIRMED'` from the booking table. This ensures the settlement is based on actual confirmed bookings, not a potentially stale event value.

The consumer is annotated with `@RabbitListener(queues = "payment.saga-listener")` at class level. It uses `@RabbitHandler` on individual methods — Spring AMQP dispatches to the correct handler based on the deserialized type of the incoming message.

5.1 onItineraryCompleted — Create Settlement & Initiate Payment

```
@RabbitHandler
public void onItineraryCompleted(ItineraryCompletedEvent event) {
    // 1. Query DB for the actual confirmed bookings sum
    Double dbSum = bookingRepository.sumConfirmedAmountByItineraryId(event.itineraryId());
    BigDecimal amount = dbSum != null ? BigDecimal.valueOf(dbSum) : BigDecimal.ZERO;

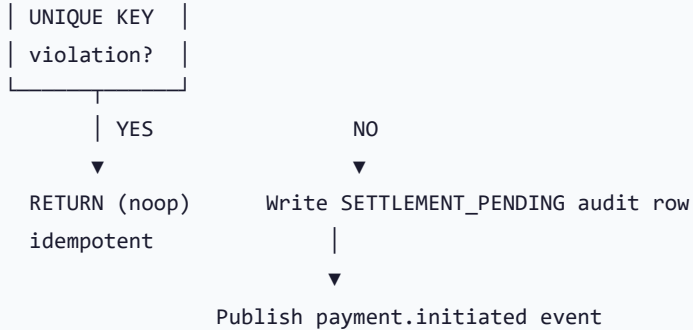
    // 2. Build and save Settlement (status = PENDING)
    Settlement settlement = new Settlement();
    settlement.setItineraryId(event.itineraryId());
    settlement.setUserId(event.organizerId());
    settlement.setAmount(amount);
    settlement.setStatus(SettlementStatus.PENDING);

    Settlement saved;
    try {
        saved = settlementRepository.save(settlement);
    } catch (DataIntegrityViolationException e) {
        // Idempotency: unique constraint on itinerary_id catches duplicate delivery
        return; // swallow silently — already processed
    }

    // 3. Write audit row: SETTLEMENT_PENDING
    paymentAuditEventRepository.save(buildAudit("SETTLEMENT_PENDING", saved));

    // 4. Publish payment.initiated
    paymentEventPublisher.publishPaymentInitiated(
        new PaymentInitiatedEvent(saved.getId(), saved.getItineraryId(), saved.getAmount())
    );
}
```





5.2 onItineraryCancelled — Cancel Bookings & Refund

```

@RabbitHandler
public void onItineraryCancelled(ItineraryCancelledEvent event) {
    List<Booking> bookings = bookingRepository.findByItineraryId(event.itineraryId());

    for (Booking b : bookings) {
        String expectedStatus = b.getStatus().name(); // "PENDING" or "CONFIRMED"
        // Atomic: UPDATE bookings SET status='CANCELLED' WHERE id=X AND status=expectedS
tatus
        int rows = bookingRepository.updateStatusByIdAndStatus(
            b.getId(), "CANCELLED", expectedStatus);

        if (rows == 0) continue; // concurrent caller already cancelled — idempotent skip

        if (b.getStatus() == BookingStatus.CONFIRMED) {
            // Write BOOKING_REFUNDED audit row
            paymentAuditEventRepository.save(buildAudit("BOOKING_REFUNDED", b));
            // Publish payment.refunded per confirmed booking
            paymentEventPublisher.publishPaymentRefunded(
                new PaymentRefundedEvent(settlementId, event.itineraryId(),
                    BigDecimal.valueOf(b.getAmount())));
        }
    }

    // Transition settlement to REFUNDED (if it exists and is not already REFUNDED)
    settlementRepository.findByItineraryId(event.itineraryId()).ifPresent(s -> {
        if (s.getStatus() != SettlementStatus.REFUNDED) {
            s.setStatus(SettlementStatus.REFUNDED);
            s.setSettledAt(LocalDateTime.now());
            settlementRepository.save(s);
        }
    });
}
  
```

IDEMPOTENCY IN ONITINERARYCANCELLED

The atomic `UPDATE ... WHERE status='EXPECTED'` is the idempotency guard. If a message is delivered twice, the second delivery finds `rowcount = 0` (already CANCELLED) and skips the audit + refund event. No double-refunds possible.

Settlement.java — JPA Entity

```

@Entity
@Table(name = "settlements")
public class Settlement {

    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(unique = true) // ← the idempotency constraint
    private Long itineraryId;

    private Long userId;
    private BigDecimal amount;

    @Enumerated(EnumType.STRING)
    private SettlementStatus status;

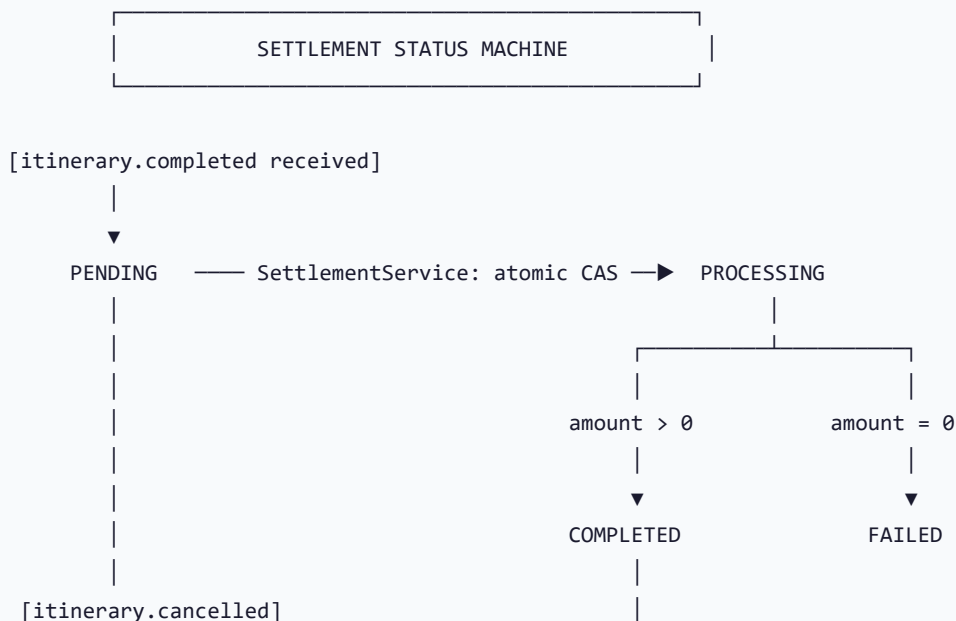
    private LocalDateTime createdAt; // set by @PrePersist
    private LocalDateTime settledAt; // set on COMPLETED/FAILED/REFUNDED
    private String failureReason;

    public enum SettlementStatus {
        PENDING, PROCESSING, COMPLETED, FAILED, REFUNDED
    }

    @PrePersist
    void onCreate() { this.createdAt = LocalDateTime.now(); }
}

```

Settlement Lifecycle State Machine





REFUNDED

SettlementRepository.java — Key Methods

```
public interface SettlementRepository extends JpaRepository<Settlement, Long> {

    Optional<Settlement> findByItineraryId(Long itineraryId);

    // SELECT FOR UPDATE — locks the row to prevent concurrent updates
    @Lock(LockModeType.PESSIMISTIC_WRITE)
    @Query("SELECT s FROM Settlement s WHERE s.itineraryId = :itineraryId")
    Optional<Settlement> findByItineraryIdForUpdate(@Param("itineraryId") Long itineraryId);

    // Atomic conditional update — returns rowcount (0 = race lost, 1 = won)
    @Modifying @Transactional
    @Query("UPDATE Settlement s SET s.status = :newStatus " +
           "WHERE s.id = :id AND s.status = :expectedStatus")
    int updateStatusIfMatches(Long id, SettlementStatus newStatus,
                              SettlementStatus expectedStatus);
}
```

WHY PESSIMISTIC_WRITE (SELECT FOR UPDATE)?

In the `SettlementService` flow, we need to read the current status and then conditionally update it. Without a row lock, two concurrent requests could both read "PENDING", both decide to proceed, and both try to set it to PROCESSING — resulting in double-processing. The lock prevents this race condition.

SettlementService — Full Idempotency Lifecycle

The `POST /settlements/process` endpoint processes a payment for a settlement. It follows a strict 6-step decision tree to ensure idempotency and correctness.

Request Body: SettlementProcessRequest

```
{ "itineraryId": 20, "userId": 1, "amount": 2000.00 }
```

The 6-Step Decision Tree

1 Load settlement with lock

Call `settlementRepository.findByItineraryIdForUpdate(itineraryId)` .

If empty → 404 Not Found

2 Inspect locked status

PROCESSING → 409 Conflict (already in flight)

COMPLETED → 200 OK return prior result (idempotent re-delivery)

FAILED or **REFUNDED** → 409 Conflict (terminal state)

3 Authorization checks

Extract userId from JWT token in `Authorization` header.

If JWT `userId` $\neq$ `request.userId` $\rightarrow$ 403 Forbidden

If request.amount $\neq$ settlement.amount $\rightarrow$ 400 Bad Request

4 Atomic transition: PENDING → PROCESSING

Call `updateStatusIfMatches(id, PROCESSING, PENDING)`.

If rowcount = 0 → 409 Conflict (concurrent caller won the race)

5 Business decision

```
approved = (amount.compareTo(BigDecimal.ZERO) > 0)
```

6 Finalize

If approved → status = **COMPLETED**, write audit, publish `payment.completed`

If not approved → status = **FAILED** , write audit, publish `payment.failed`

Return `SettlementResultDTO` with status, settlementId, itineraryId, amount

```
// Step 6b: failure path
settlement.setStatus(SettlementStatus.FAILED);
settlement.setFailureReason("Insufficient amount");
settlement.setSettledAt(LocalDateTime.now());
settlementRepository.save(settlement);
paymentEventPublisher.publishPaymentFailed(
    new PaymentFailedEvent(settlement.getId(),
                           settlement.getItineraryId(), "Insufficient amount"));
```

Three aggregate query endpoints, all backed by JPQL queries in `BookingRepository` :

Endpoint	Method	Returns
GET <code>/bookings/user/{userId}/total?start=&end=</code>	<code>getUserTotal</code>	<code>UserBookingTotalDTO</code> — total amount + trip count in date range
GET <code>/bookings/itinerary/{id}/confirmed-summary</code>	<code>getConfirmedSummary</code>	<code>ConfirmedSummaryDTO</code> — count + total revenue of CONFIRMED bookings
POST <code>/bookings/aggregate-by-itineraries</code>	<code>aggregateByItineraries</code>	<code>ItineraryAggregateDTO</code> — total bookings + revenue across list of itinerary IDs

```
// getUserTotal - validates date range, handles null DB sum
public UserBookingTotalDTO getUserTotal(Long userId, LocalDate start, LocalDate end) {
    if (start.isAfter(end))
        throw new ResponseStatusException(HttpStatus.BAD_REQUEST, "startDate after endDate");

    Double sum = bookingRepository.sumConfirmedAmountByUserAndRange(userId, start, end);
    Long count = bookingRepository.countConfirmedByUserAndRange(userId, start, end);

    return new UserBookingTotalDTO(userId,
        sum != null ? BigDecimal.valueOf(sum) : BigDecimal.ZERO,
        count != null ? count : 0L);
}

// getConfirmedSummary - gracefully handles null/empty Object[] from DB
public ConfirmedSummaryDTO getConfirmedSummary(Long itineraryId) {
    Object[] row = bookingRepository.getConfirmedSummaryByItinerary(itineraryId);
    if (row == null || row.length < 2)
        return new ConfirmedSummaryDTO(0L, BigDecimal.ZERO);

    long count = row[0] != null ? ((Number)row[0]).longValue() : 0L;
    double rev = row[1] != null ? ((Number)row[1]).doubleValue() : 0.0;
    return new ConfirmedSummaryDTO(count, BigDecimal.valueOf(rev));
}
```

Spring Cloud OpenFeign provides declarative HTTP clients. The contracts module declares the interfaces; booking-service uses them as `@Autowired` dependencies with no implementation code needed.

9.1 S5-F3: getUserBookingSummary — User Existence Check

Endpoint: `GET /bookings/user/{userId}/summary`

Before returning the booking summary, the service verifies the user exists by calling user-service via Feign:

```
public UserBookingSummaryDTO getUserBookingSummary(Long userId) {
    try {
        userServiceClient.getUser(userId); // 404 if user doesn't exist
    } catch (FeignException.NotFound e) {
        throw new ResponseStatusException(HttpStatus.NOT_FOUND, "User not found");
    }
    List<Booking> bookings = bookingRepository.findByUserId(userId);
    // ... build and return summary DTO
}
```

9.2 S5-F4: createFromRequest — Itinerary Validation + Seasonal Surcharge

Endpoint: `POST /bookings`

Creates a booking only if the itinerary is PLANNED or IN_PROGRESS. Applies a 20% surcharge if the destination has more than 5 active itineraries (peak season).

```
public Booking createFromRequest(CreateBookingRequest req) {
    // 1. Validate itinerary status via Feign
    ItineraryDTO itinerary = itineraryServiceClient.getItinerary(req.getItineraryId());
    if (!Set.of("PLANNED", "IN_PROGRESS").contains(itinerary.status()))
        throw new ResponseStatusException(HttpStatus.BAD_REQUEST, "Itinerary not bookable");

    // 2. Check peak season
    Long activeCount = itineraryServiceClient
        .getDestinationActiveCount(itinerary.destinationId());
    double price = req.getBasePrice();
    if (activeCount != null && activeCount > 5) price *= 1.20; // 20% peak surcharge

    // 3. Save booking
    Booking booking = new Booking();
    booking.setItineraryId(req.getItineraryId());
    booking.setAmount(price);
    booking.setStatus(BookingStatus.PENDING);
    return bookingRepository.save(booking);
}
```

9.3 S5-F10: getRevenueByDestinationAndSeason — Batch Feign Calls

Endpoint: `GET /bookings/analytics/revenue-by-destination`

Queries CONFIRMED bookings from local DB, then makes two batch Feign calls to enrich with destination and itinerary info, groups revenue by destination name and season.

```
public List<DestinationSeasonRevenueDTO> getRevenueByDestinationAndSeason(
    LocalDate start, LocalDate end) {

    // 1. Query local bookings
    List<Booking> confirmed = bookingRepository.findConfirmedByCreatedAtBetween(start, end);
    List<Long> itinIds = confirmed.stream().map(Booking::getItineraryId).toList();

    // 2. Batch Feign → itinerary-service (get destinationId per itinerary)
    Map<Long, ItinerarySummaryDTO> itinMap = itineraryServiceClient
        .batchGetItineraries(new BatchItineraryRequest(itinIds))
        .stream().collect(toMap(ItinerarySummaryDTO::id, identity()));

    // 3. Batch Feign → destination-service (get destination name + season)
    List<Long> destIds = itinMap.values().stream().map(i -> i.destinationId()).toList();
    Map<Long, DestinationSummaryDTO> destMap = destinationServiceClient
        .batchGetDestinations(new BatchDestinationRequest(destIds))
        .stream().collect(toMap(DestinationSummaryDTO::id, identity()));

    // 4. Group by destination + season, sum revenue
    return groupAndSum(confirmed, itinMap, destMap);
}
```

9.4 S5-F12: processRefundCancellationTier — Atomic UPDATE + Refund

Endpoint: `POST /bookings/{bookingId}/refund`

Cancels a CONFIRMED booking using an atomic conditional UPDATE (prevents double-refund).
 Publishes `payment.refunded` event.

```
public RefundResultDTO processRefundCancellationTier(Long bookingId,
    RefundCancellationRequest req) {

    // 1. Validate itinerary is cancellable
    ItineraryDTO itinerary = itineraryServiceClient.getItinerary(req.getItineraryId());
    // ... apply refund tier logic based on departure date proximity

    // 2. Atomic: UPDATE bookings SET status='CANCELLED' WHERE id=X AND status='CONFIRMED'
    int rowcount = bookingRepository.updateStatusByIdAndStatus(
        bookingId, "CANCELLED", "CONFIRMED");

    if (rowcount == 0)
        throw new ResponseStatusException(HttpStatus.CONFLICT,
            "Booking already cancelled or not CONFIRMED");

    // 3. Publish payment.refunded
    paymentEventPublisher.publishPaymentRefunded(
        new PaymentRefundedEvent(settlementId, req.getItineraryId(),
            BigDecimal.valueOf(refundAmount)));
}
```



```
return buildRefundResult(...);  
}
```

WHY ATOMIC UPDATE MATTERS HERE

If the user double-clicks "Cancel" and two concurrent HTTP requests arrive, both read booking as CONFIRMED. Without the conditional UPDATE, both would succeed and publish two refund events. With it, only the first call gets `rowcount=1` ; the second gets `rowcount=0` → 409 Conflict. Safe.

All custom queries use JPQL (not native SQL) so they work with the JPA field names (`b.itineraryId` , `b.amount`) rather than the DB column names (`itin_id` , `amount`).

```
// Sum of CONFIRMED booking amounts for a user in a date range
@Query("SELECT SUM(b.amount) FROM Booking b WHERE b.userId = :userId " +
      "AND b.status = 'CONFIRMED' AND b.createdAt BETWEEN :start AND :end")
Double sumConfirmedAmountByUserAndRange(@Param("userId") Long userId,
      @Param("start") LocalDate start, @Param("end") LocalDate end);

// Count of confirmed bookings for a user
@Query("SELECT COUNT(b) FROM Booking b WHERE b.userId = :userId " +
      "AND b.status = 'CONFIRMED' AND b.createdAt BETWEEN :start AND :end")
Long countConfirmedByUserAndRange(@Param("userId") Long userId,
      @Param("start") LocalDate start, @Param("end") LocalDate end);

// Sum of CONFIRMED amounts for one itinerary (used by consumer)
@Query("SELECT SUM(b.amount) FROM Booking b WHERE b.itineraryId = :itineraryId " +
      "AND b.status = 'CONFIRMED'")
Double sumConfirmedAmountByItineraryId(@Param("itineraryId") Long itineraryId);

// Count + sum for confirmed-summary endpoint (returns Object[] array)
@Query("SELECT COUNT(b), COALESCE(SUM(b.amount), 0) FROM Booking b " +
      "WHERE b.itineraryId = :itineraryId AND b.status = 'CONFIRMED'")
Object[] getConfirmedSummaryByItinerary(@Param("itineraryId") Long itineraryId);

// Cross-itinerary aggregate (for POST /aggregate-by-itineraries)
@Query("SELECT COUNT(b), SUM(b.amount) FROM Booking b " +
      "WHERE b.itineraryId IN :ids " +
      "AND (:start IS NULL OR b.createdAt >= :start) " +
      "AND (:end IS NULL OR b.createdAt <= :end) " +
      "AND (:status IS NULL OR b.status = :status)")
Object[] aggregateByItineraryIds(@Param("ids") List<Long> ids,
      @Param("start") LocalDate start, @Param("end") LocalDate end,
      @Param("status") Booking.BookingStatus status);

// Atomic conditional update – the idempotency key for cancellations
@Modifying @Transactional
@Query(value = "UPDATE bookings SET status = :newStatus " +
      "WHERE id = :id AND status = :expectedStatus", nativeQuery = true)
int updateStatusByIdAndStatus(@Param("id") Long id,
      @Param("newStatus") String newStatus,
      @Param("expectedStatus") String expectedStatus);
```

K8s Manifests (k8s/booking-service/)

File	Purpose
<code>deployment.yaml</code>	Deployment with readiness/liveness probes on <code>/actuator/health</code>
<code>service.yaml</code>	ClusterIP Service on port 8080, name <code>booking-service</code>
<code>configmap.yaml</code>	All env vars: <code>M3_MESSAGING_ENABLED=true</code> , Feign URLs pointing to K8s service names, DB connection strings

Key ConfigMap entries

```
M3_MESSAGING_ENABLED: "true"           # enables RabbitMQ consumer in K8s
FEIGN_USER_SERVICE_URL: "http://user-service:8081"
FEIGN_ITINERARY_SERVICE_URL: "http://itinerary-service:8082"
FEIGN_DESTINATION_SERVICE_URL: "http://destination-service:8083"
SPRING_DATASOURCE_URL: "jdbc:postgresql://postgres:5432/bookingdb"
SPRING_RABBITMQ_HOST: "rabbitmq"
```

Spring Actuator — Health & Metrics

The following endpoints are exposed via `spring-boot-actuator` :

- `GET /actuator/health` — used by K8s readiness/liveness probes
- `GET /actuator/prometheus` — Prometheus scrapes this endpoint every 15s

```
# application.yml
management:
  endpoints:
    web:
      exposure:
        include: health,info,prometheus
  endpoint:
    health:
      show-details: always
```

Prometheus Scrape Config

```
# prometheus-configmap.yaml
- job_name: 'booking-service'
  static_configs:
    - targets: ['booking-service.tripplanning.svc.cluster.local:8080']
  metrics_path: /actuator/prometheus
  scrape_interval: 15s
```

Grafana Dashboard — 3 PromQL Panels

Panel	PromQL
HTTP Request Rate (req/s)	<code>rate(http_server_requests_seconds_count{job="booking-service"}[1m])</code>
5xx Error Rate	<code>rate(http_server_requests_seconds_count{job="booking-service",status=~"5.."}[1m])</code>
P95 Latency (ms)	<code>histogram_quantile(0.95, rate(http_server_requests_seconds_bucket{job="booking-service"}[5m])) * 1000</code>

Loki Log Shipping

Configured in `logback-spring.xml` using the Loki4J appender. Logs are tagged with `service=booking-service` so Grafana/Loki dashboards can filter by service.

All tests use **pure Mockito** (`@ExtendWith(MockitoExtension.class)`) — no Spring context, no database, no RabbitMQ server required. This makes them fast and infrastructure-independent.

TOTAL: 59 TESTS / 59 PASSING / 0 FAILING

Run with: `mvn test -pl booking-service`

PaymentEventPublisherTest (5 tests)

Verifies routing key and exchange correctness for each publisher method.

- ✓ `publishInitiated` uses routing key `payment.initiated` on exchange `payment.events`
- ✓ `publishCompleted` uses routing key `payment.completed`
- ✓ `publishFailed` uses routing key `payment.failed`
- ✓ `publishRefunded` uses routing key `payment.refunded`
- ✓ All 4 publishers use the same `payment.events` exchange (verified with `times(4)`)

ItineraryEventConsumerTest (10 tests)

Tests the saga consumer for both event types.

- ✓ `completed`: creates PENDING settlement and publishes `payment.initiated` with correct fields
- ✓ `completed`: `DataIntegrityViolation` swallowed — idempotent, no publish on duplicate delivery
- ✓ `completed`: null DB sum treated as `BigDecimal.ZERO`
- ✓ `completed`: settlement amount comes from DB query, NOT `event.totalAmount`
- ✓ `cancelled`: PENDING bookings cancelled, no refund event published
- ✓ `cancelled`: CONFIRMED booking cancelled → audit row written + `payment.refunded` published
- ✓ `cancelled`: `rowcount=0` → idempotent skip, no refund published
- ✓ `cancelled`: settlement transitioned to REFUNDED with `settledAt` set
- ✓ `cancelled`: already REFUNDED settlement not saved again
- ✓ `cancelled`: 2 CONFIRMED bookings → 2 separate `payment.refunded` events + 2 audit rows

SettlementServiceTest (13 tests)

Covers all branches of the 6-step idempotency flow.

- ✓ Step 1: 404 when no settlement row exists
- ✓ Step 2a: 409 when status is PROCESSING (already in flight)
- ✓ Step 2b: 200 when status is COMPLETED — returns prior result without re-publishing
- ✓ Step 2c: 409 when status is FAILED (terminal)
- ✓ Step 2c: 409 when status is REFUNDED (terminal)
- ✓ Step 3: 403 when JWT userId does not match request.userId
- ✓ Step 3: 400 when request amount differs from settlement.amount
- ✓ Step 4: 409 when rowcount=0 (concurrent caller won the race)
- ✓ Step 6a: success — COMPLETED status, audit row written, payment.completed published
- ✓ Step 6a: published PaymentCompletedEvent carries correct settlementId + itineraryId + amount
- ✓ Step 6b: amount=0 → FAILED status, payment.failed published (not payment.completed)
- ✓ 400 when request body is null
- ✓ 400 when itineraryId is missing from request

BookingAggregateServiceTest (10 tests)

- ✓ getUserTotal: returns correct totalAmount and tripCount
- ✓ getUserTotal: 400 when startDate is after endDate
- ✓ getUserTotal: null DB sum treated as zero
- ✓ getConfirmedSummary: returns count and totalRevenue from DB
- ✓ getConfirmedSummary: count=0 and revenue=0 when no CONFIRMED bookings
- ✓ getConfirmedSummary: gracefully handles null array from DB
- ✓ aggregateByItineraries: returns correct totalBookings and totalRevenue
- ✓ aggregateByItineraries: empty itineraryIds returns zero without DB call
- ✓ aggregateByItineraries: null status treated as any-status
- ✓ aggregateByItineraries: 400 when startDate is after endDate

BookingServiceS5FeaturesTest (15 tests in nested @Nested classes)

- ✓ [F3] known user returns summary with booking count and total
- ✓ [F3] 404 when Feign throws FeignException.NotFound, no DB query made
- ✓ [F4] PLANNED itinerary creates booking at base price
- ✓ [F4] IN_PROGRESS itinerary accepted

✓ [F4] CANCELLED itinerary → 400 Bad Request

✓ [F4] activeCount > 5 → 20% peak surcharge applied

✓ [F4] activeCount = 3 → no surcharge

✓ [F10] two bookings → enriched via batch Feign calls, grouped by destination correctly

✓ [F10] empty confirmed bookings → empty list without Feign calls

✓ [F12] success: booking cancelled, payment.refunded published with correct amount

✓ [F12] 409 Conflict on concurrent refund race (rowcount=0)

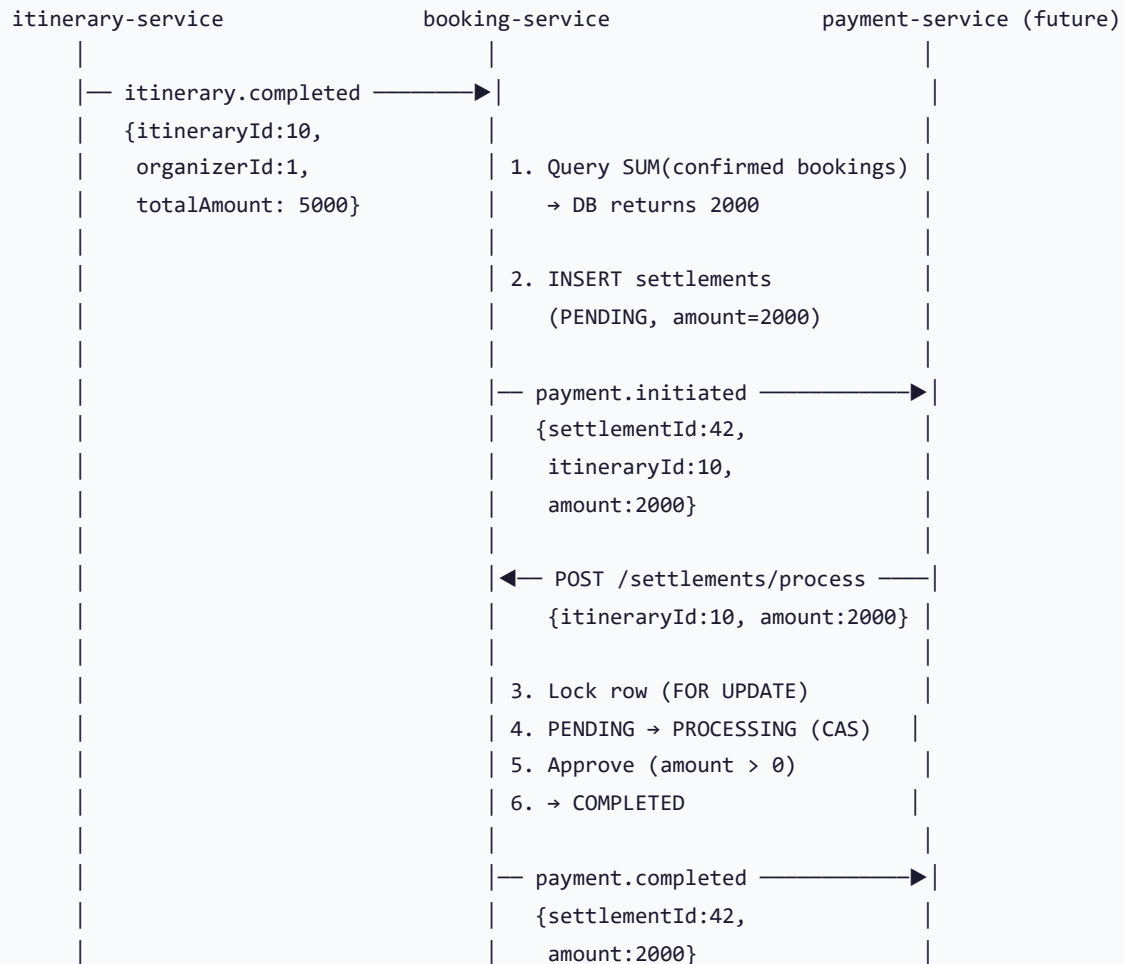
✓ [F12] 404 when itinerary not found via Feign

✓ [F12] 400 when booking is not in CONFIRMED status

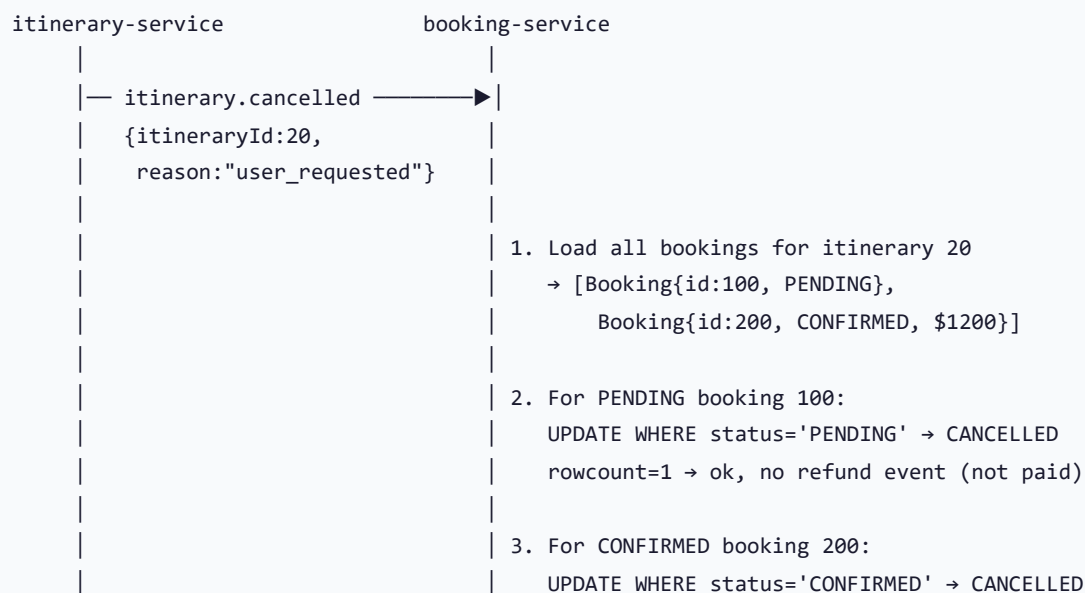
✓ [F12] refund amount correctly calculated from booking.amount

✓ [F12] audit row written on successful refund

Happy Path: Trip Completed → Payment Collected



Cancellation Path: Trip Cancelled → Refunds Issued




```

| rowcount=1 → write audit + publish refund
|
| — payment.refunded —————> (published)
| {settlementId:55,
|   itineraryId:20,
|   refundAmount:1200}
|
| 4. Settlement → REFUNDED (with settledAt)

```

Idempotency Summary Table

Scenario	Mechanism	Effect
itinerary.completed delivered twice	UNIQUE constraint on <code>settlements.itinerary_id</code>	<code>DataIntegrityViolationException</code> caught, second delivery silently ignored
POST <code>/settlements/process</code> called twice	SELECT FOR UPDATE + status check	Second call sees COMPLETED status → returns prior result immediately (200 OK)
PENDING → PROCESSING race condition	Atomic conditional UPDATE (CAS)	One caller gets rowcount=1, other gets rowcount=0 → 409 Conflict
itinerary.cancelled delivered twice	Atomic UPDATE WHERE <code>status='CONFIRMED'</code>	Second delivery: rowcount=0 → no audit, no refund event
POST <code>/bookings/{id}/refund</code> double-click	Atomic UPDATE WHERE <code>status='CONFIRMED'</code>	Second request: rowcount=0 → 409 Conflict

KEY TAKEAWAY

Every state transition that has side effects (publishing events, writing audit rows) is guarded by an atomic conditional UPDATE that acts as a Compare-And-Swap (CAS). If rowcount = 0, the operation is a no-op. This guarantees exactly-once side effects even with at-least-once message delivery from RabbitMQ.